ICS0022 Secure Programming
Checkpoint 2 - CLI Password Manager

Student: Danila Sisov

Implemented scope

I have implemented the core of a local, single-user command-line password manager. Entries contain a label, username, password and optional note. They persist between runs in ~/.securepm/vault.json. The code, dependency file, run instructions, tests and progress note are stored in the project GitHub repository.

COMMAND	IMPLEMENTED BEHAVIOUR
init	Confirm a new master password and create a vault without overwriting an existing one.
add	Authenticate, prompt for an entry and save it encrypted.
list	Authenticate and display entry IDs and labels only.
show ENTRY_ID	Authenticate and display entry metadata; the password remains [hidden].

Every command accessing an existing vault requests the master password again. Password input does not echo, and the program aborts if hidden input is unavailable. The earlier proposal to reveal passwords with show was changed to masking to meet the Checkpoint 2 requirement.

Encryption, authentication and file handling

Argon2id derives a 32-byte key using a random 16-byte salt, 65536 KiB of memory, 3 passes and 4 lanes. AES-256-GCM encrypts all entry fields with a fresh 12-byte nonce on each save and a full 16-byte authentication tag. The technical header is authenticated as associated data. Successful tag verification unlocks the vault; no master password, encryption key or separate password verifier is stored on disk.

The implementation validates the JSON structure, field sizes, IDs and cryptographic settings. It rejects invalid input and unsafe file metadata. The private directory uses mode 0700; vault, lock and encrypted temporary files use 0600. A persistent advisory lock coordinates commands. Updates sync encrypted temporary data before atomic replacement. A wrong password or failed authentication tag reveals no entries and causes no save.

Environment, tests and execution

Development environment: **Arch Linux, fish, Python 3.14.6 and cryptography 50.0.2.** The dependency version is pinned. The project includes 15 automated tests covering encrypted round trips, persistence, authentication failures, tampering, input validation, permissions, links, locking, failed writes and password masking. From the repository root:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m password_manager --help
.venv/bin/python -m password_manager init
.venv/bin/python -m unittest discover -s tests -v
```

Remaining work and limitations

update, delete, change-master, generate and application logging remain future work. MacOS remains a target platform, but native macOS validation has not been performed. CPython cannot guarantee complete erasure of secrets from memory. Password recovery and rollback detection are not provided.